Translating your Simulink mathematical model into C++ for your NodeMCU is where the real engineering happens! 🛠️

Since a full Fast Fourier Transform (FFT) can be heavy for a standard microcontroller, we will use a highly efficient **Digital Signal Processing (DSP) trick** called an Exponential Moving Average (EMA) to act as our Low-Pass Filter. Then, we will calculate the dynamic variance (energy) of the signal to detect those heavy, low-frequency elephant footsteps (seismic AI). 🔗

Here is your step-by-step guide and the complete C++ code. 🚀

1. Wiring Guide for NodeMCU

Before uploading the code, ensure your hardware is wired correctly.

MPU6050 (I^2C Protocol):

- **VCC:** 3.3V
- **GND:** GND
- **SDA:** D2 (GPIO4)
- **SCL:** D1 (GPIO5)

Ra-02 LoRa Module 433MHz (SPI Protocol):

- **VCC:** 3.3V *(Do not use 5V!)*
- **GND:** GND
- **MISO:** D6 (GPIO12)
- **MOSI:** D7 (GPIO13)
- **SCK:** D5 (GPIO14)
- **NSS (CS):** D8 (GPIO15)
- **RST:** D0 (GPIO16)
- **DIO0:** D3 (GPIO0)

---

2. Required Arduino Libraries

Open your Arduino IDE, go to **Sketch > Include Library > Manage Libraries**, and install these:

1. **Adafruit MPU6050** (by Adafruit)
2. **Adafruit Unified Sensor** (by Adafruit)
3. **LoRa** (by Sandeep Mistry - crucial for the Ra-02)

---

3. The C++ Edge AI Code

Copy and paste this into your Arduino IDE. Read through the comments to see how your Simulink logic translates to C++! 📘

C++

```cpp
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <SPI.h>
#include <LoRa.h>

// --- LoRa Pin Definitions for NodeMCU ---
#define LORA_SS    15 // D8
#define LORA_RST   16 // D0
#define LORA_DIO0  0  // D3

Adafruit_MPU6050 mpu;

// --- DSP & Filter Variables ---
float filtered_Z = 0.0;
float baseline_Z = 0.0;
const float ALPHA = 0.2; // Low-Pass Filter constant (lower = smoother/lower frequency)

// --- Classification Variables ---
const float IMPACT_THRESHOLD = 1.5; // Change this based on your real-world testing!
unsigned long lastAlertTime = 0;
const unsigned long ALERT_COOLDOWN = 5000; // Wait 5 seconds between LoRa alerts

void setup() {
  Serial.begin(115200);
  while (!Serial) delay(10);

  Serial.println("Initializing Pathfinder Seismic Node...");

  // 1. Initialize MPU6050
  if (!mpu.begin()) {
    Serial.println("Failed to find MPU6050 chip! Check wiring.");
    while (1) { delay(10); }
  }
  mpu.setAccelerometerRange(MPU6050_RANGE_2_G);
  mpu.setFilterBandwidth(MPU6050_BAND_44_HZ); // Hardware-level low pass filter!
  Serial.println("MPU6050 Found!");

  // 2. Initialize LoRa Ra-02
  LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
  // Set frequency to 433MHz for Ra-02 module
  if (!LoRa.begin(433E6)) {
    Serial.println("Starting LoRa failed! Check wiring.");
    while (1);
  }
  // Optimize LoRa for long range, low data rate
  LoRa.setSpreadingFactor(10);
  LoRa.setSignalBandwidth(62.5E3);
  Serial.println("LoRa Initialized Successfully!");

  // Calibrate baseline Z-axis gravity (usually ~9.8 m/s^2)
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);
  baseline_Z = a.acceleration.z;
}

void loop() {
  // 1. Read Raw Sensor Data
  sensors_event_t a, g, temp;
  mpu.getEvent(&a, &g, &temp);

  float raw_Z = a.acceleration.z - baseline_Z; // Remove baseline gravity

  // 2. Apply Software Low-Pass Filter (EMA)
  // Translating the Simulink discrete filter to C++
  filtered_Z = (ALPHA * raw_Z) + ((1.0 - ALPHA) * filtered_Z);

  // Calculate the absolute impact energy (Bandpower equivalent)
  float impact_energy = abs(filtered_Z);

  // Print to Serial Plotter for visual debugging
  Serial.print("Raw_Z:"); Serial.print(raw_Z); Serial.print(",");
  Serial.print("Filtered_Z:"); Serial.print(filtered_Z); Serial.print(",");
  Serial.print("Threshold:"); Serial.println(IMPACT_THRESHOLD);

  // 3. Classification Logic (Decision Gate)
  if (impact_energy > IMPACT_THRESHOLD) {

    // Check if cooldown has passed to avoid spamming the network
    if (millis() - lastAlertTime > ALERT_COOLDOWN) {
      Serial.println("🚨 CLASSIFICATION: Heavy Impact Detected! (Potential Elephant)");

      // 4. Transmit SOS / Alert via LoRaWAN backbone
      sendLoRaAlert();

      lastAlertTime = millis();
    }
  }

  // Sample roughly at 100Hz
  delay(10);
}

// Function to handle LoRa transmission
void sendLoRaAlert() {
  Serial.println("📡 Transmitting LoRa Alert...");

  LoRa.beginPacket();
  // Transmitting specific emergency string formatting
  LoRa.print("NODE_01, ALERT: SEISMIC_HEC, IMPACT:");
  LoRa.print(filtered_Z);
  LoRa.endPacket();

  Serial.println("✅ Transmission Complete.");
}
```

4. How to Test It Live 🩺

1. Upload the code to your NodeMCU.
2. Open the **Serial Plotter** (Tools > Serial Plotter) in the Arduino IDE. Set the baud rate to 115200.
3. You will see three lines: Raw Data, Filtered Data, and your Threshold line.
4. **The Test:** Lightly tap your desk with your finger (simulating high-frequency noise like a vehicle). The raw data will spike, but the filtered data will stay low.
5. Now, **thump the table heavily with your fist** (simulating a low-frequency elephant step). You should see the filtered data spike past the threshold, and the Serial Monitor will confirm the LoRa transmission! 🐘

Would you like to move on to the **LabVIEW Dashboard design** so you can receive and visualize these LoRa alerts on your computer, or would you prefer to jump into the **PCB Design Guide**?